Lab 35: Two linked defects go in one detailed request

Topic 3: Claude Code Configuration & Workflows · Task 3.5: Apply iterative refinement techniques for progressive improvement

Runs on Simulator 3: Multi-Agent Research System · Self-practice

Source: SkillCertPro practice exam Part 1, Q27 (unofficial third-party question)

Commit to an answer first

After a refactor, generated reports show two linked defects: citations render twice in the final report, and some evidence excerpts disappear when the synthesis subagent's output is passed to the report generator. The likely cause spans the claim–source schema, the synthesis prompt, and the report template, and the acceptance test should confirm that each claim appears once with its evidence excerpt and source URL. How should this team use Claude Code to address the defects most effectively?

A. Enter plan mode and give one detailed request covering both defects, affected areas, constraints, and verification checks before approving edits.

B. Add a Stop hook that runs the acceptance test and rely on repeated blocks as the main debugging strategy.

C. Skip planning and ask Claude to fix the report bugs broadly, then inspect the diff after it finishes.

D. Open separate cleared sessions for each defect, ask each session to edit only its assigned file, then merge results.

Simulate the solution

```
# In the browser: open Simulator 3, then Lab 35
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 35

# In a terminal, from simulators/simulator-3-multi-agent-research/
uv run research_system.py --case UC-27
```

What you should see

You write one plan-mode request: both defects, shared interface, acceptance test.

A plan-mode request with all five parts.

Open `simulators/simulator-3-multi-agent-research/evidence/` and find it yourself.

The answer

A. Plan mode with one request naming both defects and the checks.

Match the workflow to how the work depends on itself. These fixes interact, so splitting them early lets each session assume something different about the shared interface between synthesis and report generation. A strong request describes the symptoms, likely affected areas, constraints, what “fixed” means and how to verify it, and asks Claude to run the checks and iterate. Plan mode is the right start because the change is uncertain and spans several files.

Why the others fail

B. Stop hooks strengthen verification, but they complement diagnosis rather than replace it.

C. A broad, unplanned fix across three coupled files is likely to make inconsistent edits.

D. /clear and separate sessions suit unrelated tasks or polluted context, not coupled defects that need a shared mental model.

Check yourself

- request names both defects and the shared interface
- includes a verification command (acceptance test)
- explains why split sessions (D) and no-plan (C) are weaker

Now change one thing

In fixtures/sources.json replace the visual-arts TIMEOUT with [] and rerun: a valid empty result must not read as a failure.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.